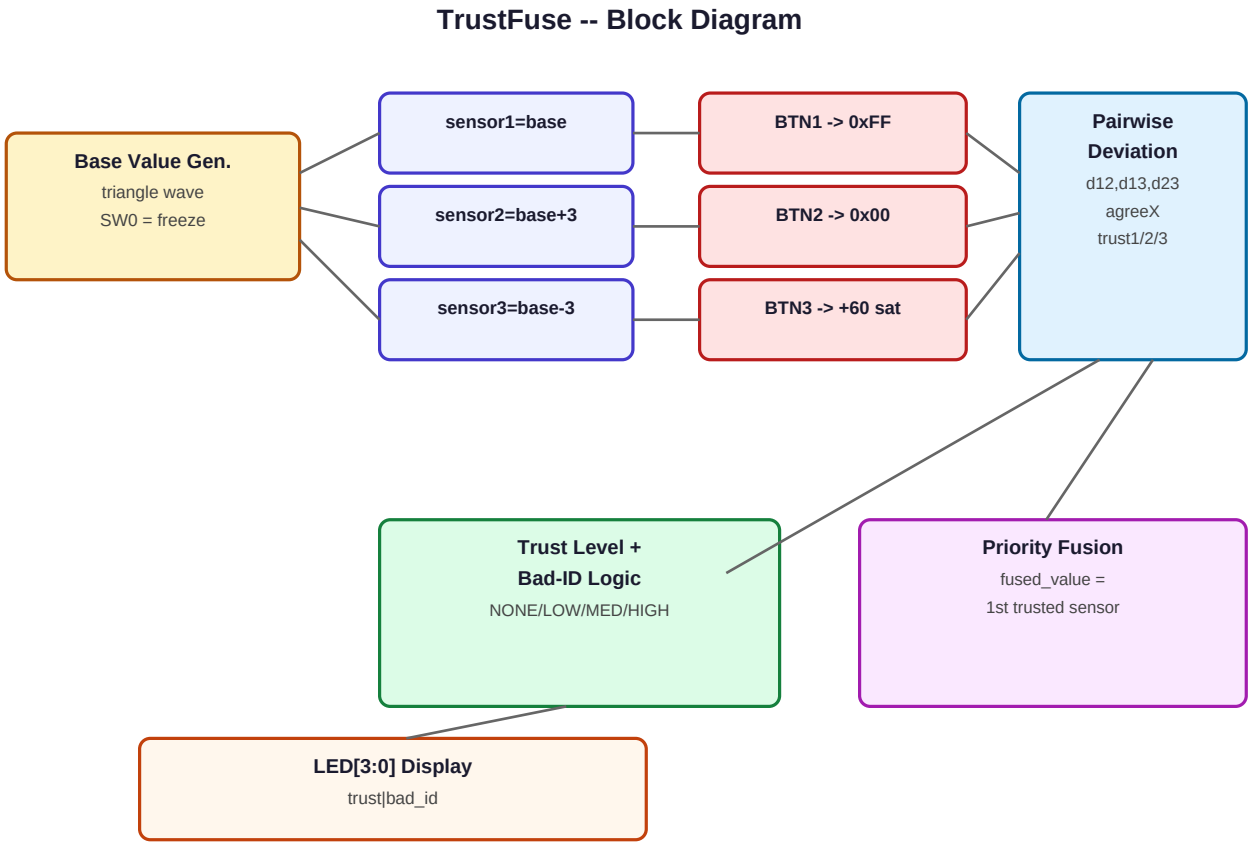
TrustFuse — Hardware Sensor Trust

V-SPACE FPGA Build Challenge 2026 Team: Mahasena | **Team Lead:** Manaswi | **Experiment 2 — Beginner Board:** PYNQ-Z2 (Xilinx Zynq XC7Z020CLG400-1) | **Tool:** AMD Vivado 2026.1

1. Objective

Real embedded and safety-critical systems rarely trust a single sensor: a sensor can drift, fail stuck, or report noise. **TrustFuse** is a small hardware circuit that models three sensors measuring the *same* physical quantity, continuously cross-validates them against one another, computes a live **trust level**, identifies *which* sensor (if any) is the outlier, and produces a fault-tolerant **fused output** — entirely in combinational/synchronous logic, with no software involved. Push-buttons let a user inject a live fault on any one sensor to demonstrate the detection working in real time.

2. Block Diagram



See Images/block_diagram.svg.

Data flow: a simulated **Base Value Generator** (triangle wave) stands in for a real physical quantity. Three **sensor models** read it with small, known calibration offsets (`+0`, `+3`, `–3`). **Fault-injection muxes**, driven

by push-buttons, can force any one sensor to an obviously wrong value for demonstration. A **pairwise deviation** block computes `|sensor_i − sensor_j|` for all three pairs and compares each to a tolerance. **Trust logic** turns the three pairwise agreements into a 0–3 trust count and identifies a single outlier if exactly one exists. A **priority fusion** block selects the first trusted sensor as the fused output. The result is encoded on the 4 board LEDs.

---

3. RTL Design

3.1 Module hierarchy

```
top_module   (single module — sensor models, fault injection, deviation
              checking, trust logic, and LED encoding; Beginner scope)
```

3.2 Verilog code structure

Sensor models (nominal, before any fault):

```
wire [7:0] sensor1_nom = base_value;
wire [7:0] sensor2_nom = base_value + OFFSET;   // OFFSET = 3
wire [7:0] sensor3_nom = base_value - OFFSET;
```

Fault injection — saturating, so an injected fault always reads as clearly out-of-range instead of silently wrapping around:

```
wire [8:0] s3_fault_full = {1'b0, sensor3_nom} + 9'd60;
wire [7:0] sensor3_fault = s3_fault_full[8] ? 8'hFF : s3_fault_full[7:0];

wire [7:0] sensor1 = fault1 ? 8'hFF         : sensor1_nom;
wire [7:0] sensor2 = fault2 ? 8'h00         : sensor2_nom;
wire [7:0] sensor3 = fault3 ? sensor3_fault : sensor3_nom;
```

Pairwise agreement & trust:

```
wire agree12 = (d12 <= TOLERANCE);
wire agree13 = (d13 <= TOLERANCE);
wire agree23 = (d23 <= TOLERANCE);

wire trust1 = agree12 | agree13;   // sensor1 confirmed by at least one other
wire trust2 = agree12 | agree23;
wire trust3 = agree13 | agree23;

wire [1:0] num_trusted = trust1 + trust2 + trust3;
```

Priority fusion (no divider needed — a correct average of an arbitrary subset of 1–3 values needs a divide-by-3, which is unnecessary complexity for a value with no on-board numeric display; a priority mux is exact, cheap, and easy to verify by hand):

```
wire [7:0] fused_value = trust1 ? sensor1 :
                         trust2 ? sensor2 :
                         trust3 ? sensor3 :
                                  8'h00;
```

Full source: RTL/top_module.v, constraints: RTL/constraints.xdc.

3.3 LED encoding

LED[3:2] trust level	Meaning	LED[1:0]	Meaning
`11`	HIGH — all 3 agree	`01`	sensor1 is the lone outlier
`10`	MEDIUM — 1 outlier	`10`	sensor2 is the lone outlier
`01`	LOW — only 1 usable	`11`	sensor3 is the lone outlier
`00`	NONE — total fault	`00`	n/a (0, 1, or 3 sensors untrusted)

---

4. Simulation Results

Simulated with the **Vivado Simulator (xvlog / xelab / xsim)**, standalone CLI flow. Testbench: `Testbench/tb_top.v`. Raw transcript: `Simulation/transcript.txt`. Waveform database: `Simulation/tb_top_sim.wdb`.

Test strategy: the simulated base reading is pinned at a known value (`base_value == 50`, via `SW0`/`freeze`) so every expected LED code is a **hand-calculated, exact value** — with `sensor1=50, sensor2=53, sensor3=47`:

Phase	Scenario	d12 / d13 / d23	Expected LED	Result
1	No fault	3 / 3 / 6 (all ≤ 8)	`1100` (HIGH)	**PASS**
2	fault1 → sensor1=0xFF	202/208/6	`1001` (MED, sensor1 bad)	**PASS**
3	fault2 → sensor2=0x00	50/3/47	`1010` (MED, sensor2 bad)	**PASS**
4	fault3 → sensor3=107 (sat.)	3/57/54	`1011` (MED, sensor3 bad)	**PASS**
5	fault1 + fault2 together	all > tol	`0000` (NONE)	**PASS**
6	Reset	—	`base_value` → 10	**PASS**

Actual transcript output (2026-10-10):

```
================================================
 TrustFuse - Hardware Sensor Trust - Testbench
================================================

Base value pinned at 50 (sensor1=50 sensor2=53 sensor3=47)

-- Phase 1: Nominal (no fault) --
```

```
  [PASS] All three sensors agree -> HIGH trust : led=1100 (expected 1100)

-- Phase 2: Fault injected on sensor1 --
  [PASS] sensor1 forced to 0xFF -> MEDIUM trust, sensor1 flagged : led=1001 (expected 1001)

-- Phase 3: Fault injected on sensor2 --
  [PASS] sensor2 forced to 0x00 -> MEDIUM trust, sensor2 flagged : led=1010 (expected 1010)

-- Phase 4: Fault injected on sensor3 --
  [PASS] sensor3 driven to +60 (saturated) -> MEDIUM trust, sensor3 flagged : led=1011 (expected 1011)

-- Phase 5: Faults on sensor1 AND sensor2 simultaneously --
  [PASS] Two sensors faulted -> NONE trust (total fault) : led=0000 (expected 0000)

-- Phase 6: Reset behaviour --
  [PASS] Reset returns base_value to BASE_MIN (10).

================================================
 TESTBENCH SUMMARY: ALL CHECKS PASSED (0 errors)
================================================
```

■ **TO DO:** — Manaswi: open `Simulation/tb_top_sim.wdb` in the Vivado GUI waveform viewer, take a screenshot of `sensor1/2/3`, `trust_level`, `bad_id`, `led` over time, and save it as `Images/waveform.png`.

---

5. Hardware Implementation

■ **TO DO:** — pending, same license-tier issue as Experiment 1: switch Vivado to the WebPACK/BASIC license tier (Help → Manage License), then run `vivado -mode batch -source run_vivado_full.tcl` from the repo root to produce `Bitstream/top_module.bit`.

■ **TO DO:** — Manaswi: after programming the board, add:] - `Images/board_setup.jpg` — PYNQ-Z2 wired/powered and ready - `Images/hardware_output.jpg` — LEDs showing HIGH trust (`1100`), then a photo with BTN1 held showing MEDIUM trust + sensor1 flagged (`1001`) - Resource utilization report (optional): `Simulation/impl_utilization.rpt`

---

6. Applications

- **Autonomous systems / avionics**: redundant IMU, GPS, or altitude sensors are routinely cross-checked this way before being trusted by a flight/drive controller. - **Industrial safety systems**: triple-redundant pressure/temperature sensors in plant control use the same agree-by-majority principle (a lightweight relative of Triple Modular Redundancy voting). - **Automotive**: wheel-speed or battery-cell voltage sensors are fused with outlier rejection for safety-rated ECUs. - **Fault-tolerant IoT**: a cheap way to add graceful degradation — the system keeps working with 2 good sensors, and clearly reports when it can no longer trust any reading.

---

7. Conclusion

TrustFuse demonstrates a complete, hardware-only sensor-fusion and fault-detection pipeline, verified exhaustively in simulation: the nominal case, each of the three single-sensor faults, and a total double-fault case all produced exactly the hand-predicted LED code, with **zero simulation errors**.

A known limitation, found and documented honestly during design: with pairwise tolerance-based voting, if a faulted sensor's forced value happens to coincidentally land close (within `TOLERANCE`) to one of the *other* sensors' current reading, the algorithm can be fooled into still trusting it. This was observed by hand-calculation near `base_value ≈ 10` (where a sensor forced to `0` can land within tolerance of a nominal `~7` reading) — which is exactly why the testbench deliberately validates the fault scenarios at a safer mid-range operating point (`base_value = 50`). This is a genuine, textbook property of simple threshold-based voting schemes, and documenting it is itself a key learning outcome of this experiment.

Future scope: add a *persistence* check (a sensor must disagree for several consecutive ticks before being flagged, rejecting single-sample noise spikes) and a true weighted/median fusion instead of priority selection — both natural extensions this team plans to explore in the **FAULTX – HIL Fault Injection** and **Adaptive PUF Authentication** experiments later in this challenge.

Document prepared for the V-SPACE FPGA Build Challenge 2026 — Team Mahasena.